The React Native Guide

Created by JS Mastery

What's in the Guide?

Most people try to learn React Native the wrong way.

They watch hours of tutorials, build half-finished apps, and still feel stuck when it’s time to build something on their own.

This guide fixes that.

Instead of throwing random topics at you, it shows you the exact path from basics → real apps → getting hired, in a way that actually makes sense.

By the end of this guide, you’ll:

- understand what React Native is and how it works at a high level
- know the exact prerequisites you actually need (and what you don’t)
- have a clear, step-by-step roadmap to follow
- understand how to approach learning without getting stuck
- know how to tell when you’re actually job-ready
- avoid the most common mistakes that slow people down
- build better habits from the start
- know exactly what to focus on next

> **Think of this as a roadmap, not a textbook.**

You don’t need to memorize everything here.

You just need to follow it, build along the way, and improve step by step.

If you stick with it and actually build things, you won’t just understand React Native — you’ll be able to ship real apps.

Introduction

React Native lets you build real mobile apps using JavaScript and React.

Instead of writing separate apps for iOS and Android, you write one codebase that runs on both.

You describe UI → React Native turns it into native components.

That means:

- your buttons are real native buttons
- your scroll feels like a real app
- your app isn’t a website wrapped in a shell

Why people use it

- faster development (one codebase)
- huge ecosystem (React + JS) & strong community
- works well with tools like Expo

Prerequisites

This is where most people either struggle quietly… or quit.

Let’s make it clear.

1.1 JavaScript (What you ACTUALLY need)

You don’t need “advanced JS”. But you do need to be comfortable, not confused.

Must understand deeply

1. Variables & Data Types

- string
- number
- boolean
- arrays
- objects

2. Functions

- arrow functions
- passing functions as props

3. Arrays (VERY important)

You will use most of these daily:

- `.map()`
- `.filter()`
- `.find()`
- `.some()`
- `.every()`
- `.reduce()`

4. Objects (You’ll use these constantly)

Most real data is not simple values. It’s objects.

- accessing values
- updating values
- spreading
- nested objects (real-world case)

- destructuring (cleaner code)

5. Async JavaScript (CRITICAL)

You must understand:

- `async/await`
- promises

> **Reality check:** If async code confuses you, APIs will feel impossible.

6. Truthy / Falsy

Used everywhere in UI.

7. Optional Chaining

Prevents crashes.

8. Basic Error Handling

Very underrated.

9. Modules

Import / Export.

10. Spread & Rest Operators

Used daily.

1.2 React Fundamentals (Non-negotiable)

1. Components

Everything is a component.

2. Props

Passing data.

3. State (Core idea)

State = data that changes UI.

> **Analogy:** State = memory of your app.

4. useEffect (Most misunderstood)

- API calls
- side effects

1.3 TypeScript (Don’t skip in 2026)

You don’t need advanced types.

Just know the basic stuff.

Why it matters

- catches bugs early
- makes code easier to understand

> **Simple idea:** TypeScript = labels for your data.

1.4 Git (Basic survival skills)

You don’t need to be a Git expert.

But you do need to be comfortable enough to not panic when things go wrong.

The Core Flow

You’ll use this daily:

- pulling changes
- working with others
- branches (this is where real work happens)
- merging (bringing work together)
- viewing history & undoing mistakes (very important)
- good commit messages
- working with GitHub (real-world flow)

> **Reality:** If you can’t push code, you can’t work in a team.

1.5 CLI Comfort

You’ll use the terminal daily:

- run apps
- install packages
- debug issues

> **Tip:** Don’t fear the terminal. It’s just buttons without UI.

1.6 What You DON’T Need (yet)

- Swift or Objective-C
- Kotlin or Java (Android native)
- Writing native modules from scratch
- Complex system design
- Low-level performance optimization

Roadmap

This is the part most people want, but few follow properly.

Phase 1 — Foundation

Goal

Be comfortable building basic screens.

Learn

- React basics (state, props, hooks)
- Core components:

- View → layout container
- Text → text rendering
- Image → images
- Pressable → buttons / interactions

- Styling (flexbox)
- SafeAreaView
- Scroll & Lists (Early exposure) → ScrollView, FlatList

Build

Simple UI app

- header
- list
- buttons

Rule: Don’t just read. Build immediately.

Phase 2 — Navigation + Layout

Learn

- Expo Router
- file-based routing
- stack navigation
- tab navigation

Build

Multi-screen app

- Home screen
- Details screen
- Back navigation

Analogy: Navigation = pages in a book.

Phase 3 — Data + APIs

Learn

- fetch / axios
- async/await
- useEffect for data fetching
- loading states
- error handling

Introduce

- Basic Pagination / Load More
- Basic Data Transformation
- React Query

Build

API-based app

- fetch data
- loading state
- error handling
- list display
- empty state
- “pull to refresh” (bonus)

This is a big milestone: You’re now building real apps.

Phase 4 — State Management

Learn

- prop drilling (problem to notice)
- useState vs global state
- Zustand
- Server State (React Query)

Build

App with shared state

- user login state
- saved/bookmarked items

- UI reacting to state changes

Analogy: Global state = shared memory across screens.

Phase 5 — Forms + Auth

Learn

- input handling
- react-hook-form
- validation (important)
- auth flow (big picture)
- protected Routes

Build

Auth app

- login/signup
- validation
- token storage
- protected screens
- logout

Phase 6 — Native Features

Learn

- camera
- permissions
- notifications
- storage

Build

Feature-rich app

- send notification
- image upload
- permission handling
- local storage
- small haptic feedback

Now it feels like a real mobile app.

Phase 7 — Animations + Polish

Learn

- why animations matter
- basic animations
- micro-interactions
- react-native-reanimated
- gestures

Build

Polished UI

- smoother transitions
- better loading states
- consistent UI
- better UX

This is what impresses recruiters.

Phase 8 — Performance + Cleanup

Learn

- why performance matters
- FlatList optimization
- memoization
- avoiding unnecessary re-renders
- remove unused code
- consistent file structure
- error handling (app-wide)

Improve

Existing apps

- remove lag
- clean UI
- optimize lists
- clean code

Phase 9 — Deployment

Learn

- Expo EAS (main tool)

Understand

- APK vs AAB
- submit to store
- generate build
- install on your phone

Milestone: Your app is now live.

Phase 10 — Final Portfolio Project

Build ONE strong app

Must include:

- auth
- API
- good UI
- animations
- real use case

Examples

- habit tracker
- finance app
- AI tool app

This is your “hire me” project.

How to Know You’re Ready

You’re ready when

- you can build an app without a tutorial
- you can debug your own bugs
- you can fetch data from an API & display it properly
- you can deploy an app
- you understand data flow
- you’ve built 2–3 complete apps

You’re not ready when

- you still rely on tutorials for basic features

- you copy code without understanding it
- your app breaks and you don’t know where to start

Simple test: Can you build something from scratch in 2–3 days?

If yes → you’re ready.

Common Traps

1. Tutorial addiction

Watching ≠ learning.

2. Avoiding bugs

Bugs = learning.

3. Skipping deployment

Huge mistake.

4. Overengineering

Keep things simple.

5. Not finishing projects

Finish small apps first.

6. Chasing too many tools

Stick to: React Native, Expo, and a few solid tools.

Key Takeaways

- Learn JS + React properly first
- Use Expo (default in 2026)
- Build while learning
- Focus on real apps
- Learn deployment early
- One strong project > many weak ones

Final Thought

You don’t need to know everything.

You need to:

- understand the basics well

- build real things
- fix your mistakes

Do that consistently, and you'll be ahead of most people trying to learn this.

That's It

You now have the full roadmap.

Not just what to learn… but what actually matters.

If you've gone through this properly, you should:

- understand how React Native works
- know what to focus on (and what to ignore)
- have a clear path from beginner → job-ready
- feel confident building real apps

Now the only thing left is execution.

Build. Break things. Fix them. Repeat.

What's Next?

This guide gives you the path.

But if you want:

- step-by-step builds
- real production apps
- deeper explanations (without confusion)
- patterns used in real companies

Join the Waitlist of the React Native Course

A complete, practical program to take you from:

"I kinda get it" → "I can build and ship real apps confidently"

No fluff. No endless theory. Just what actually makes you better.

The Ultimate React Native Course

After carefully considering your passion for developing full-stack mobile applications from scratch and diving into every aspect of it, we're excited to let you know that your dream of learning and building such apps is within reach.

We're working on a PRO course that will teach you:

- Understanding the inner workings of React Native
- Mastering React Native fundamentals with a focus on every detail
- Creating pixel-perfect UIs for both iOS and Android devices

- Developing apps with Expo Go and Emulators
- Exploring Expo Router with best practices
- Fetching APIs using hooks and TanStack Query
- Building complex forms with Zod and React Hook Form
- Implementing authentication and authorization from scratch (no third-party tools)
- Utilizing the latest Expo features, including RSCs and Expo API routes
- Creating a custom backend with Node.js and Express.js
- Managing databases with MongoDB and Postgres with Prisma
- Understanding and using native modules
- Animating with React Native Reanimated
- Handling payments with Stripe and Razorpay
- Conducting unit testing
- Implementing real-time push notifications, messaging, deployment, and more

If this sounds like what you're looking for, join us quickly to stay updated on course details and suggest any features you'd like to see (we value your input).

You can reply to the email from which you received this guide, or drop your suggestion in the suggestions channel.

Plus, you'll have the opportunity to receive a special discount just for you.